Inference-Time Hallucination Detection in Vision-Language Models

Project Overview Document

Om Sanjaykumar Patel
Master of Science in Artificial Intelligence Systems
Herbert Wertheim College of Engineering
University of Florida

Spring 2026

Project Summary

This project builds an instrumented inference pipeline for Vision-Language Models (VLMs) with integrated, training-free hallucination detection. Three detection methods—token entropy, visual contrastive decoding, and attention-based grounding—are evaluated on standard benchmarks (POPE, CHAIR) under varying quantization levels and sustained inference workloads. A Streamlit dashboard provides interactive result exploration. The system runs on consumer GPUs (RTX 4060, 8GB VRAM) using INT4 quantization.

Model:	LLaVA-1.5-7B (frozen, no fine-tuning)
Hardware:	RTX 4060 (8GB) / HiPerGator A100
Benchmarks:	POPE, CHAIR, Confidence Drift
Language:	Python 3.10 + PyTorch 2.2
Dashboard:	Streamlit + Plotly

Contents

1 Background: What Are Vision-Language Models? 2
- 1.1 Definition . . . 2
- 1.2 How VLMs Work . . . 2
- 1.3 Key VLMs in 2025–2026 . . . 2
- 1.4 Why VLMs Matter . . . 2

2 The Problem: VLM Hallucination 2
- 2.1 What Is Hallucination? . . . 2
- 2.2 Why Do VLMs Hallucinate? . . . 3
- 2.3 Why This Matters . . . 3

3 Why Inference-Time Detection? 3
- 3.1 Training-Time Approaches (What Exists) . . . 3
- 3.2 The Case for Inference-Time Detection . . . 4
- 3.3 What Is Missing in the Literature . . . 4

4 Project Approach 4
- 4.1 System Architecture . . . 4
- 4.2 Detection Methods . . . 5
  - 4.2.1 Method 1: Token-Level Entropy . . . 5
  - 4.2.2 Method 2: Visual Contrastive Decoding . . . 5
  - 4.2.3 Method 3: Attention-Based Grounding . . . 5

5 Data and Benchmarks 6
- 5.1 POPE: Polling-based Object Probing Evaluation . . . 6
- 5.2 CHAIR: Caption Hallucination Assessment with Image Relevance . . . 6

6 Project Structure and Pipeline 7
- 6.1 File Organization . . . 7
- 6.2 Execution Pipeline . . . 7
- 6.3 What `python run.py full` Executes . . . 7

7 Technology Stack 8

8 Hardware Configuration 8

9 Expected Outputs 8
- 9.1 Generated Plots . . . 8
- 9.2 Streamlit Dashboard Tabs . . . 9

10 References 9

1 Background: What Are Vision-Language Models?

1.1 Definition

A Vision-Language Model (VLM) is a neural network that can simultaneously process **visual inputs** (images, video frames) and **textual inputs** (questions, instructions) to produce coherent text outputs. Unlike traditional computer vision models that output bounding boxes or class labels, VLMs generate free-form natural language—answering questions about images, describing scenes, and reasoning about visual content.

1.2 How VLMs Work

A typical VLM consists of three components:

1. **Vision Encoder:** A pre-trained image model (e.g., CLIP ViT-L/14) that converts an image into a sequence of visual feature vectors called *image tokens*. For a 336×336 image with 14×14 patches, this produces 576 image tokens.
2. **Projection Layer:** A small neural network (typically a 2-layer MLP) that maps vision encoder outputs into the same embedding space as the language model's text tokens.
3. **Language Model:** A pre-trained large language model (e.g., Vicuna-7B, LLaMA) that receives the projected image tokens concatenated with the text tokens and generates an answer autoregressively—one token at a time.

1.3 Key VLMs in 2025–2026

Model	Organization	Size	Notable Features
LLaVA-1.5	UW–Madison	7B/13B	Open-source, instruction-tuned, widely used baseline
Qwen2-VL	Alibaba	2B–72B	Dynamic resolution, strong multilingual
LLaMA 3.2 Vision	Meta	11B/90B	Integrated vision natively into LLaMA
Gemini 2.5 Pro	Google	—	1M+ token context, state-of-the-art reasoning
GPT-4.1	OpenAI	—	Strong OCR, chart understanding, visual math

Table 1: Major Vision-Language Models as of 2026

1.4 Why VLMs Matter

VLMs are being deployed in medical imaging (radiology report generation), autonomous driving (scene understanding), document analysis (invoice/form parsing), accessibility (image descriptions for visually impaired users), and retail (product cataloging). The global AI market is projected to reach $391 billion in 2025, and VLM research now accounts for approximately 40% of accepted papers at top venues (CVPR, ICLR, NeurIPS).

2 The Problem: VLM Hallucination

2.1 What Is Hallucination?

In the context of VLMs, **hallucination** refers to the generation of text that is plausible and fluent but **not supported by the visual input**. Examples:

Hallucination Examples

Image: A park bench with trees in the background.
Question: "Is there a dog in this image?"
Model answer: "Yes, there is a small dog sitting on the bench." ← *Hallucinated*

Image: A kitchen with a refrigerator, oven, and sink.
Prompt: "Describe this image in detail."
Model answer: "The kitchen features a microwave above the stove..." ← *Hallucinated (no microwave)*

2.2 Why Do VLMs Hallucinate?

VLM hallucinations are fundamentally different from text-only LLM errors. They arise from a **cross-modal misalignment**: the language decoder stops attending to image features and instead generates text based on statistical language patterns. Three primary causes:

1. **Language prior dominance:** The language model has seen "kitchen" associated with "microwave" millions of times during pre-training. It generates "microwave" because it is statistically likely, not because it sees one.
2. **Under-attended image regions:** The model's attention disperses across the image without focusing on specific objects, causing it to miss what is (or is not) actually present.
3. **Vision-language misalignment:** The projection layer imperfectly maps visual features into the language model's embedding space, creating "blind spots" where visual evidence is lost.

2.3 Why This Matters

Hallucination is not a minor annoyance—it is a **safety-critical failure** in deployment:

- **Medical imaging:** A VLM describing a "mass in the left lung" that does not exist could trigger unnecessary invasive procedures.
- **Autonomous driving:** Hallucinating a pedestrian could cause emergency braking; missing a real one could cause a collision.
- **Legal/financial documents:** Hallucinated text in invoice parsing or contract analysis can cause material financial harm.
- **Accessibility:** Blind users relying on VLM image descriptions receive fabricated information about their environment.

3 Why Inference-Time Detection?

3.1 Training-Time Approaches (What Exists)

The majority of hallucination mitigation research focuses on interventions during training:

Approach	Description	Limitation
DPO/RLHF	Train on preference pairs (hallucinated vs. grounded)	Expensive, model-specific
Contrastive SFT	Fine-tune on hallucination-aware data	Requires curated datasets
Reward models	Train a separate critic to score outputs	Doubles compute cost
Data augmentation	Add negative examples to training data	Does not generalize

Table 2: Training-time hallucination mitigation approaches

The core problem: all training-time methods require modifying model weights. In production, you often deploy a frozen, pre-trained model that you cannot retrain—due to cost, data access, or licensing

constraints.

3.2 The Case for Inference-Time Detection

Inference-time methods work on **frozen models** with **no retraining**. They extract detection signals from the model's own outputs—logits, attention weights, and distribution shifts—to flag unreliable generations at the point of delivery.

Advantages: zero training cost, model-agnostic (works on any VLM), immediately deployable, can be added as a monitoring layer on top of existing serving infrastructure.

3.3 What Is Missing in the Literature

Despite dozens of papers on VLM hallucination detection in 2024–2025, three critical questions remain unanswered:

1. **Operational robustness:** How do detection methods behave under quantization (INT4/INT8 vs. FP16)? No study systematically varies serving conditions.
2. **Temporal stability:** Do hallucination patterns shift over 1000+ consecutive inferences? No benchmark captures confidence drift.
3. **Practical integration:** Can multiple detection signals be combined into a deployable monitoring system? Papers report metrics in isolation.

This project addresses all three.

4 Project Approach

4.1 System Architecture

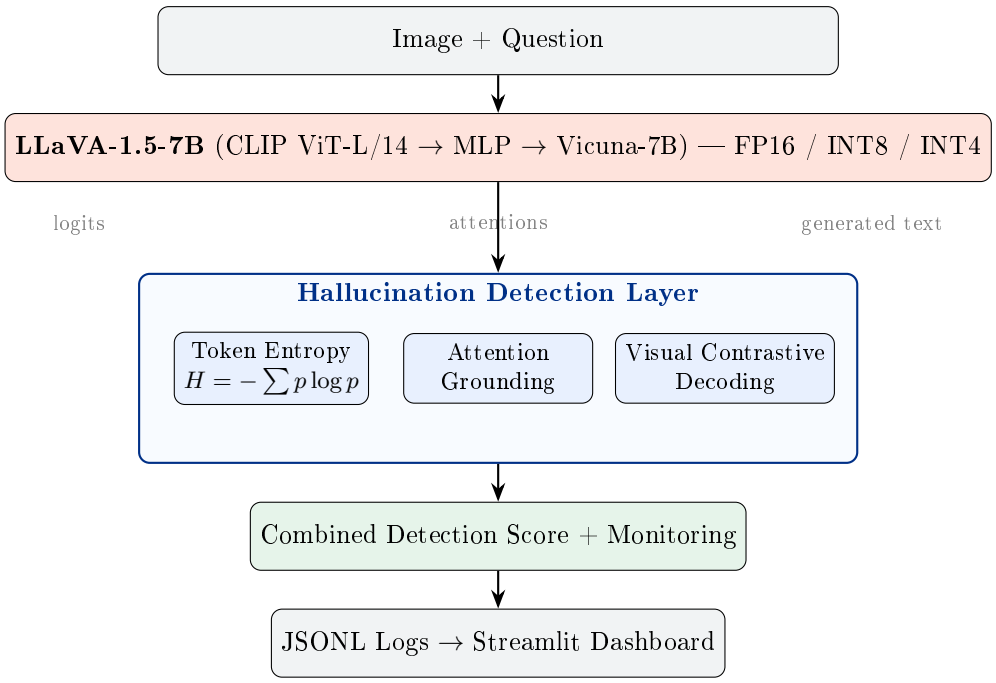

Figure 1: System architecture: instrumented VLM inference pipeline with hallucination detection

4.2 Detection Methods

4.2.1 Method 1: Token-Level Entropy

For each generated token, compute the entropy of the softmax distribution over the vocabulary:

$$H(t) = -\sum_{i=1}^{V} p_i \cdot \log p_i$$

where V is the vocabulary size and p_i is the probability of token i at step t.

Intuition: High entropy means the model is uncertain—the probability mass is spread across many tokens. This uncertainty correlates with hallucination. Low entropy means the model is confident and likely grounded.

Aggregation: Mean entropy, max entropy, and fraction of tokens above a calibrated threshold produce a per-response hallucination score.

4.2.2 Method 2: Visual Contrastive Decoding

Run inference twice:

1. With the **original image** → logit distribution P_{orig}
2. With a **perturbed image** (Gaussian noise, grayscale, or low-resolution) → logit distribution P_{pert}

Compute the KL divergence at each step:

$$D_{KL}(P_{orig} \| P_{pert}) = \sum_i P_{orig}(i) \cdot \log \frac{P_{orig}(i)}{P_{pert}(i)}$$

Intuition: If the output barely changes when the image is degraded, the model was not using visual information—it was generating from language priors. Low KL divergence signals hallucination risk.

4.2.3 Method 3: Attention-Based Grounding

In LLaVA's self-attention, the first 576 token positions correspond to image tokens. For each generated token, measure:

$$r_{visual} = \frac{\sum_{j \in image} \alpha_j}{\sum_{j \in all} \alpha_j}$$

where α_j is the attention weight from the generated token to position j.

Intuition: If the model is not attending to image tokens when generating object names, it is hallucinating from language priors. This method is disabled on GPUs with <16GB VRAM due to memory requirements.

5 Data and Benchmarks

Dataset	Type	Size	Purpose
MS-COCO val2014	Images	40,504 images	Source images for all evaluations
POPE	Yes/No QA	3,000/split	Object hallucination probing (random, popular, adversarial)
CHAIR	Captions	200–500	Free-form caption hallucination (object-level and sentence-level)
Drift logs	Time series	1,000+	Per-inference confidence, latency, and memory tracking

Table 3: Datasets and benchmarks used in this project

5.1 POPE: Polling-based Object Probing Evaluation

POPE asks binary yes/no questions about whether a specific object exists in an image. Three splits test increasing difficulty:

- **Random:** Negative objects sampled randomly from COCO categories.
- **Popular:** Negative objects are the most frequent categories in COCO (harder—model has strong language priors for common objects).
- **Adversarial:** Negative objects frequently co-occur with the ground-truth objects in the image (hardest—tests whether the model can resist co-occurrence bias).

5.2 CHAIR: Caption Hallucination Assessment with Image Relevance

CHAIR evaluates free-form captions by extracting mentioned objects and comparing against ground-truth COCO annotations:

- $\mathbf{CHAIR}_i$ = (hallucinated objects) / (all mentioned objects) — object-level rate
- $\mathbf{CHAIR}_s$ = (sentences with $\geq$1 hallucination) / (total sentences) — sentence-level rate

6 Project Structure and Pipeline

6.1 File Organization

```
vlm_project/
  run.py                                          Single entry point for everything
  dashboard.py                                    Streamlit interactive results viewer
  requirements.txt
  configs/experiment_config.yaml                  All hyperparameters
  scripts/
    utils.py                                      Model loading, I/O, monitoring
    01_download_data.py                           COCO + POPE + model cache
    02_basic_inference.py                         Single inference test
    03_batched_pipeline.py                        Throughput benchmarking
    04_entropy_detector.py                        Detection method 1
    05_attention_detector.py                      Detection method 2
    06_contrastive_decoder.py                     Detection method 3
    07_pope_evaluation.py                         POPE benchmark runner
    08_chair_evaluation.py                        CHAIR evaluation
    09_confidence_monitor.py                      Drift tracking
    10_experiment_runner.py                       Full experiment matrix
    11_analysis_plots.py                          Figure generation
  outputs/                                        All results, logs, and plots
```

6.2 Execution Pipeline

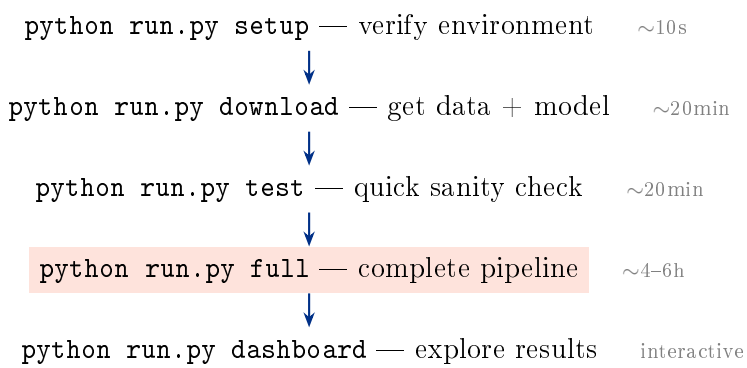

Figure 2: Five-command workflow from setup to results exploration

6.3 What `python run.py full` Executes

Step	Script	What It Does	Time
1	01_download_data	Download COCO val2014, POPE, cache LLaVA	20 min
2	02_basic_inference	Verify model loads and generates text	2 min
3	03_batched_pipeline	Measure throughput at INT4, batch=1	5 min
4	10_experiment_runner	POPE: 1000 samples × 3 splits, entropy + contrastive	2–3 h
5	08_chair_evaluation	Generate + evaluate 200 image captions	30–45 min
6	09_confidence_monitor	1000 consecutive inferences with drift logging	1 h
7	11_analysis_plots	ROC curves, entropy scatter, drift charts, heatmaps	1 min

Table 4: Full pipeline breakdown (RTX 4060, INT4)

7 Technology Stack

Component	Technology	Role
Model	LLaVA-1.5-7B (HuggingFace)	VLM backbone (frozen, no fine-tuning)
Vision encoder	CLIP ViT-L/14@336px	Image feature extraction (576 tokens)
Language model	Vicuna-7B	Autoregressive text generation
Framework	PyTorch 2.2 + Transformers 4.40	Model loading, inference, attention extraction
Quantization	bitsandbytes (NF4, double quant)	INT4/INT8 compression for 8GB VRAM
Evaluation	pycocotools, scikit-learn	CHAIR metrics, ROC/AUC computation
Visualization	Matplotlib, Seaborn, Plotly	Static plots + interactive dashboard
Dashboard	Streamlit	Browser-based result exploration
Monitoring	psutil, torch.cuda	GPU memory, CPU, latency tracking

Table 5: Technology stack

8 Hardware Configuration

	Laptop (Primary)	HiPerGator (UF)
GPU	RTX 4060 (8GB VRAM)	A100 (80GB VRAM)
CPU	Intel i7 14th gen	AMD EPYC
RAM	16GB	32GB+
Quantization	INT4 only	FP16 / INT8 / INT4
Batch size	1	1 / 8 / 32
Attention detector	Disabled (OOM)	Enabled
POPE samples	1,000/split	3,000/split

Table 6: Hardware-adaptive configuration

The system auto-detects available VRAM and adjusts: if FP16 is requested on an 8GB GPU, it automatically downgrades to INT4 with a warning instead of crashing.

9 Expected Outputs

9.1 Generated Plots

The analysis pipeline generates six publication-ready figures:

1. **ROC Curves:** Per-POPE-split receiver operating characteristic for each detection method, with AUC scores.
2. **Quantization Comparison:** Bar chart of accuracy and hallucination rate across FP16/INT8/INT4.
3. **Entropy Analysis:** Histogram of entropy distributions separated by hallucination status; scatter plot of entropy vs. top-1 probability.
4. **Confidence Drift:** Four-panel time series showing entropy, latency, GPU memory, and rolling accuracy over 1000+ inferences.
5. **Detection Heatmap:** AUC values across (quantization $\times$ POPE split) as a color-coded matrix.
6. **LaTeX Table:** Copy-paste-ready results table for the final report.

9.2 Streamlit Dashboard Tabs

Tab	Contents
Overview	Accuracy, hallucination rate, CHAIR scores, sample counts
POPE Results	Interactive ROC curves, entropy histograms, filterable sample browser
CHAIR Results	Most hallucinated objects bar chart, caption browser with error highlighting
Confidence Drift	Smoothable time-series (entropy, latency, GPU memory, accuracy) with first/second half stats
Detection Comparison	Side-by-side AUC bars, entropy vs. contrastive scatter, per-split breakdown

Table 7: Streamlit dashboard contents

10 References

[1] H. Liu et al., "Visual Instruction Tuning," *NeurIPS*, 2023. (LLaVA)

[2] H. Liu et al., "Improved Baselines with Visual Instruction Tuning," 2023. (LLaVA-1.5)

[3] Y. Li et al., "Evaluating Object Hallucination in Large Vision-Language Models," *EMNLP*, 2023. (POPE)

[4] A. Rohrbach et al., "Object Hallucination in Image Captioning," *EMNLP*, 2018. (CHAIR)

[5] V. Prabhakaran et al., "VADE: Visual Attention Guided Hallucination Detection and Elimination," *ACL Findings*, 2025.

[6] "Reducing Hallucinations in Large Vision-Language Models via Latent Space Steering," *ICLR*, 2025. (VTI)

[7] "ECD: Efficient Contrastive Decoding with Probabilistic Hallucination Detection," 2025.

[8] "MARINE: Mitigating Hallucinations via Image-Grounded Guidance," *ICML*, 2025.

[9] "DASH: Detection and Assessment of Systematic Hallucinations of VLMs," *ICCV*, 2025.

[10] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," *ICML*, 2021. (CLIP)

[11] "Vision Language Models: A Survey of 26K Papers," *arXiv:2510.09586*, 2025.

[12] "A Survey on Efficient Vision-Language-Action Models," *arXiv:2510.24795*, 2026.

[13] "HaluGate: Token-Level Truth for Production LLMs," *vLLM Blog*, 2025.

[14] T. Dettmers et al., "QLoRA: Efficient Finetuning of Quantized Language Models," *NeurIPS*, 2023.